

D&A - row 5 (2019)

- 1) $\rightarrow$ 2 stacks of numbers - min elem. is on top of both
 $\rightarrow$ create + return a sorted stack (min. elem. on top)

function mergeStacks(s_1, s_2) is

result $\leftarrow$ init()

aux $\leftarrow$ init()

while not isEmpty(s_1) and not isEmpty(s_2) ex

if top(s_1) $\leq$ top(s_2) then

$x \leftarrow$ top(s_1)

pop(s_1)

else

$x \leftarrow$ top(s_2)

pop(s_2)

while not isEmpty(s_1) execute

$x \leftarrow$ top(s_1)

pop(s_1)

push(aux, x)

while not isEmpty(s_2) execute

$x \leftarrow$ top(s_2)

pop(s_2)

push(aux, x)

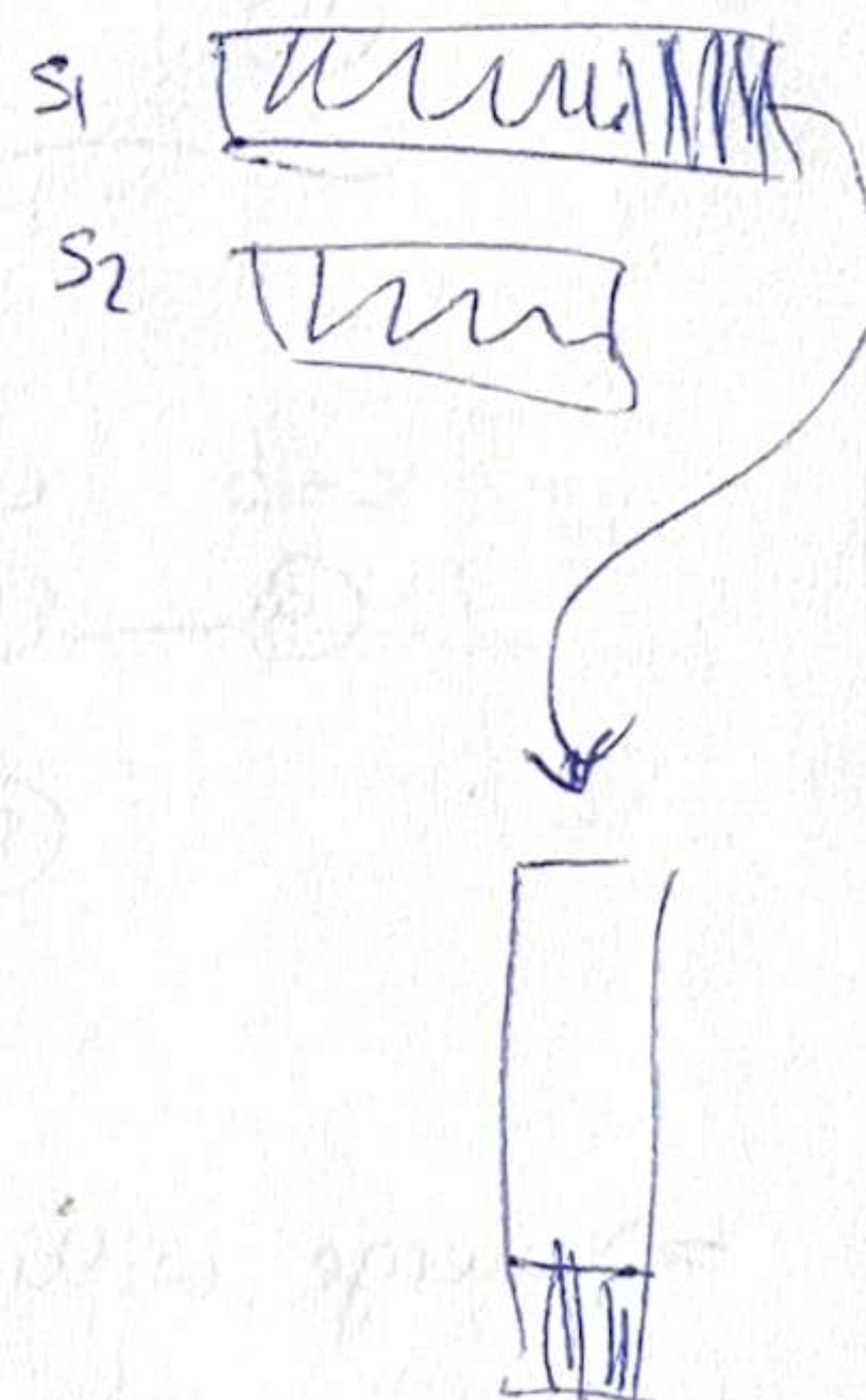
while not isEmpty(aux) execute

$x \leftarrow$ top(aux)

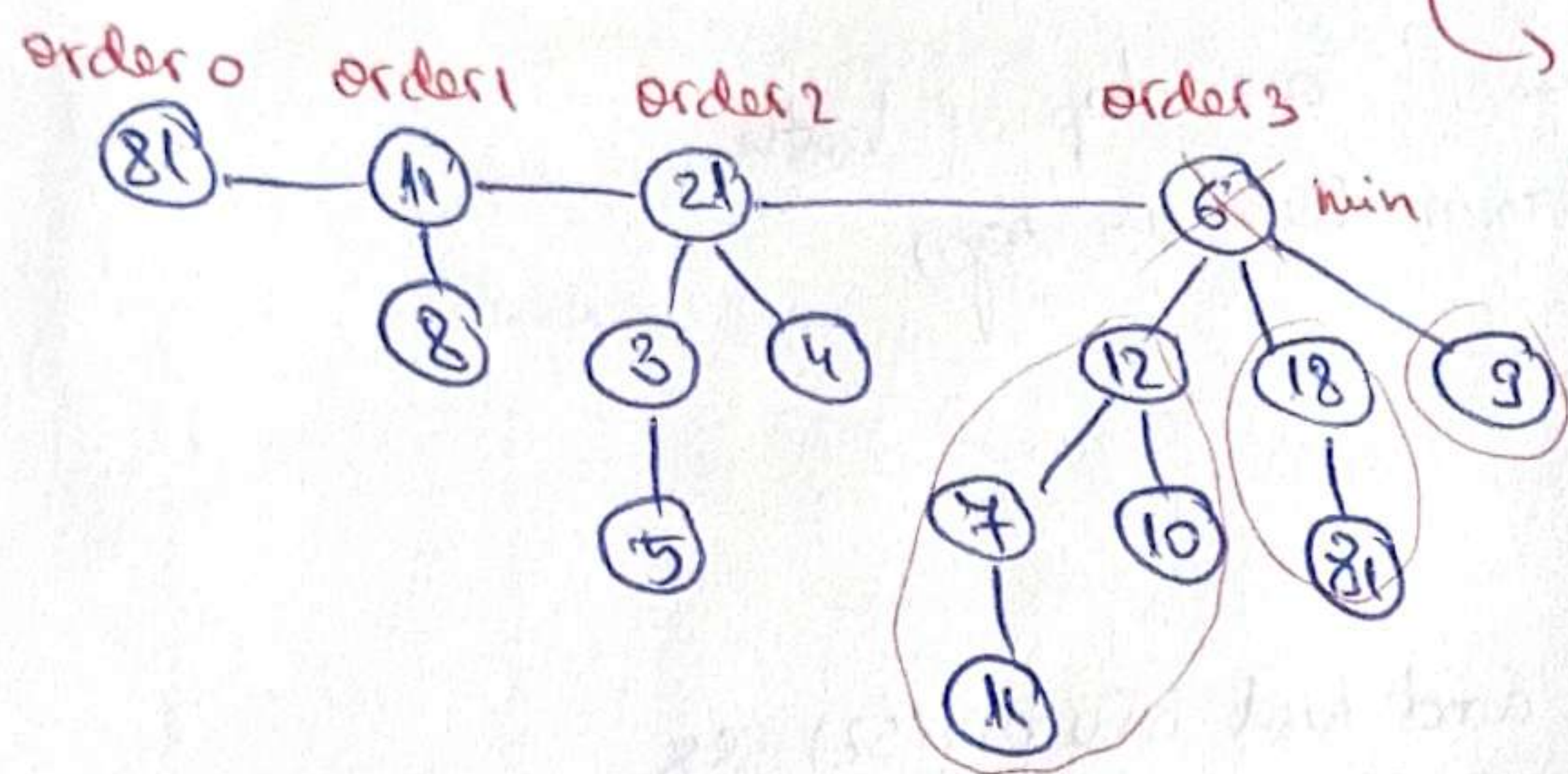
pop(aux)

push(result, x)

mergeStacks $\leftarrow$ result



2. a) Priority queue on a binomial heap. = a collection of binomial trees ($1, 2, \dots$)
pop 1 elem.

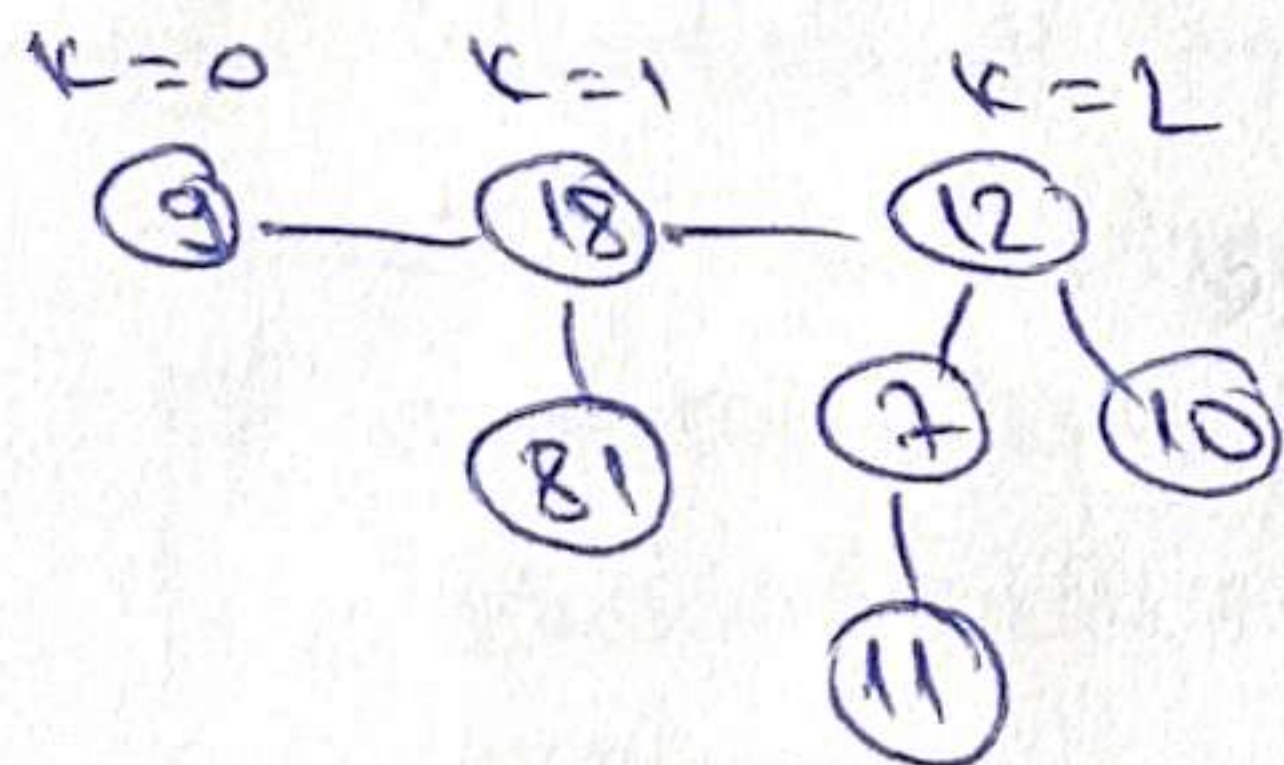
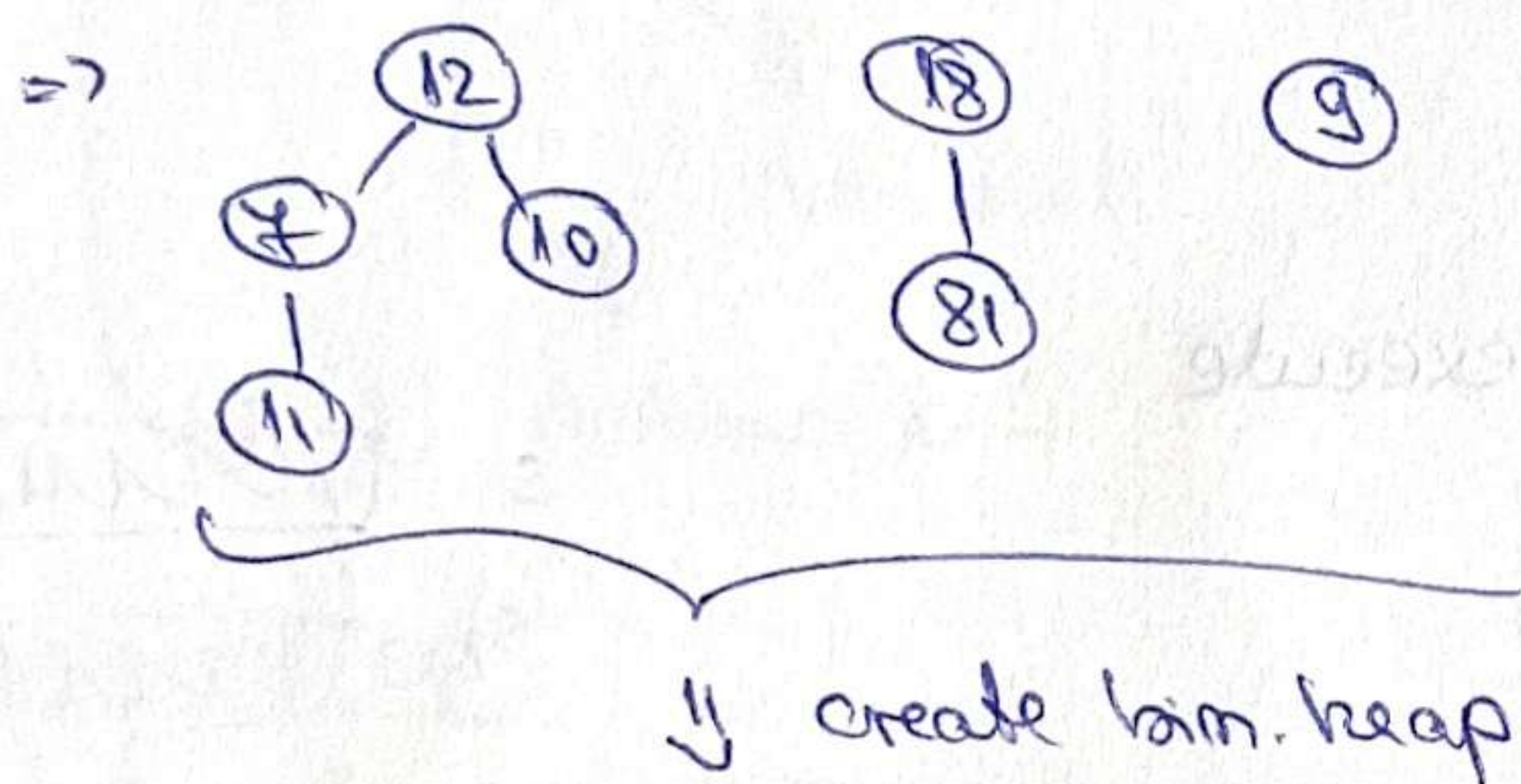


↳ merges efficiently
↳ order $k \Rightarrow 2^k$ nodes

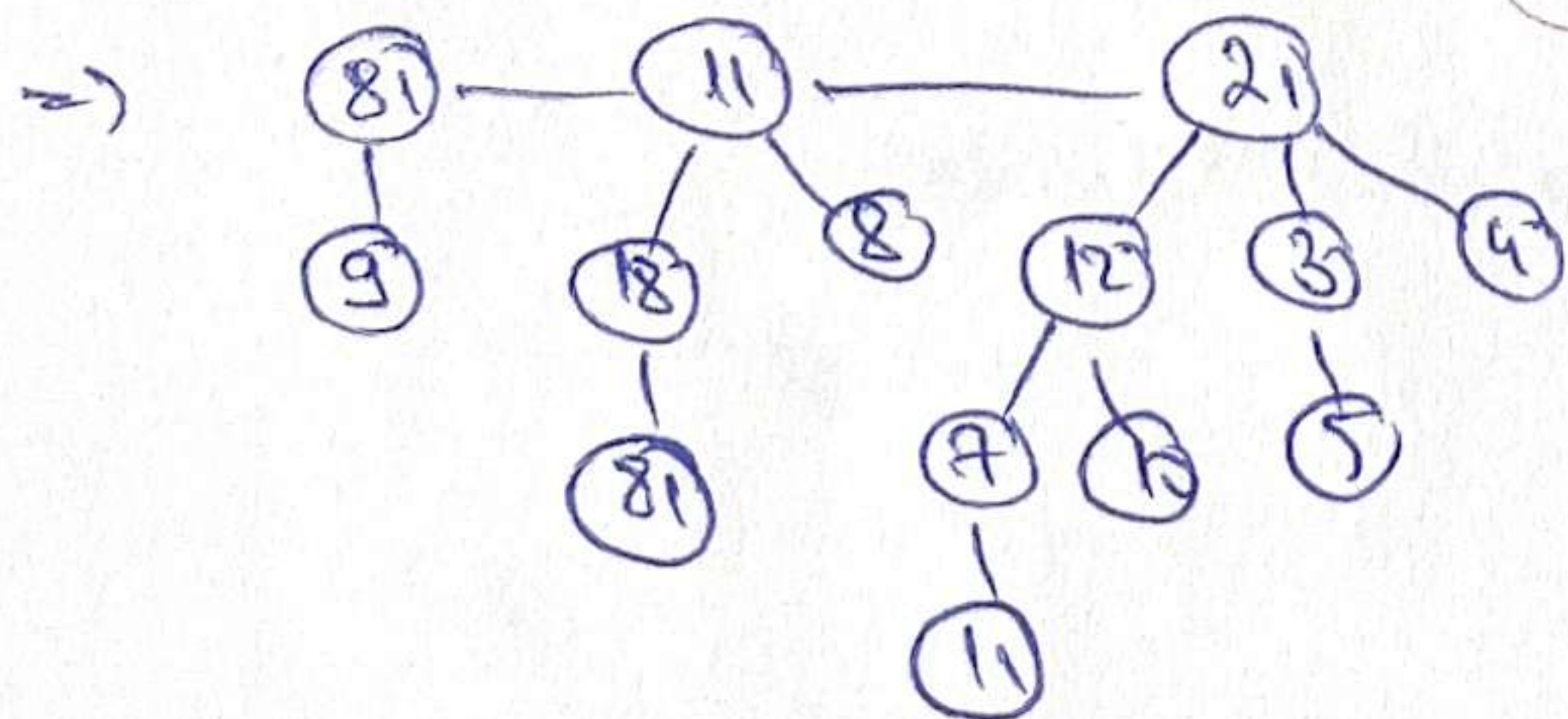
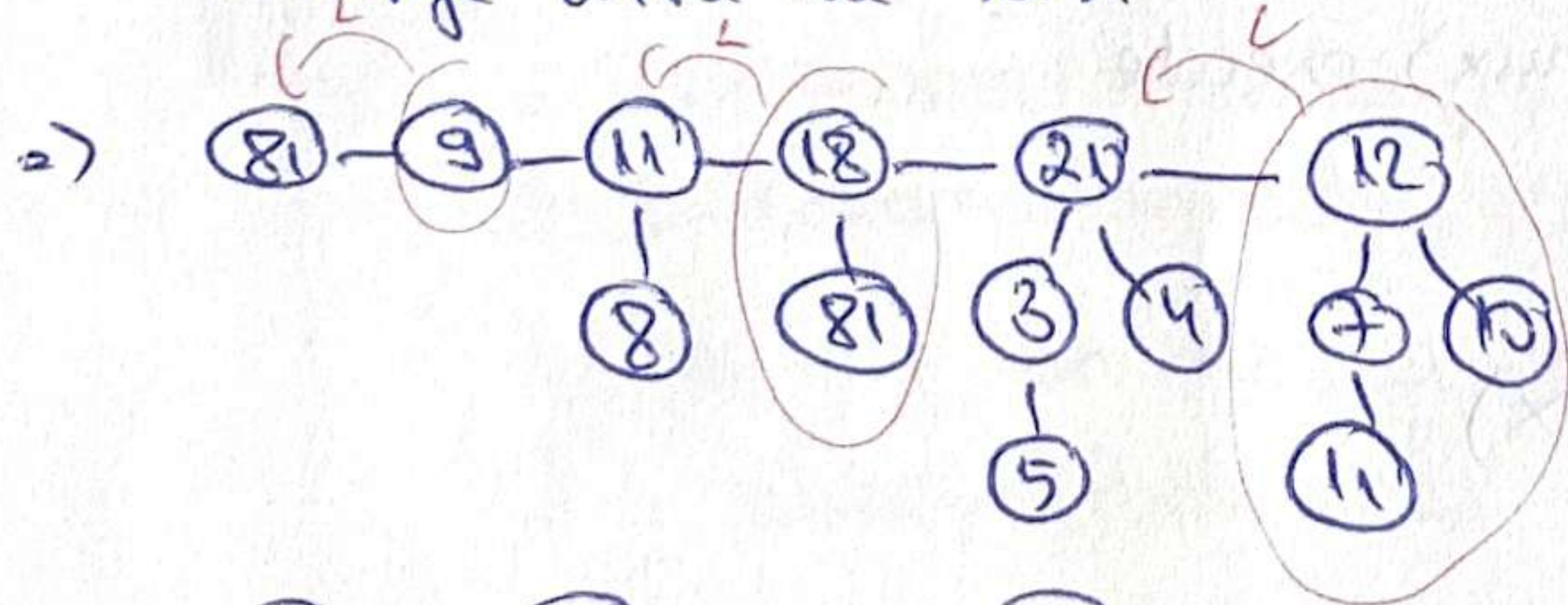
↳ delete root $\Rightarrow k$ bin. trees.

↳ 2 trees w the same order can be merged into a binomial tree of order $k+1$ by setting one to be the LEFT child of the other.

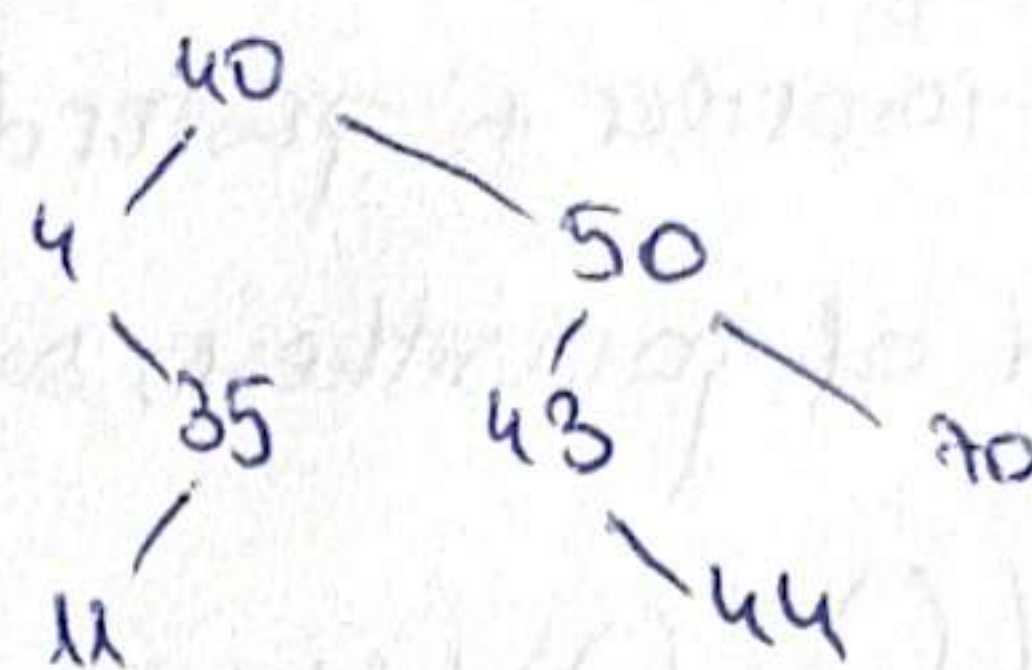
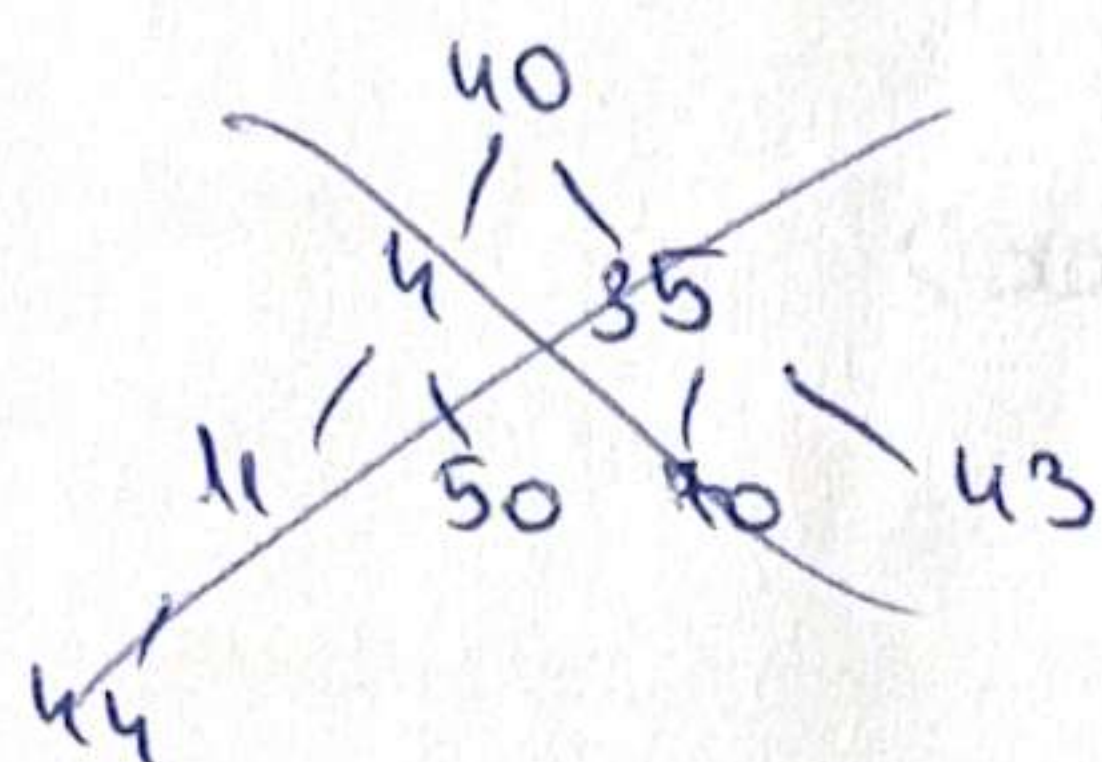
Pop from binomial heap = remove root (min) $\Rightarrow$ seq. of binomial trees
 $\Rightarrow$ transform in bin. heap $\Rightarrow$ merge with the rest of the heap.



$\Rightarrow$ merge with the rest

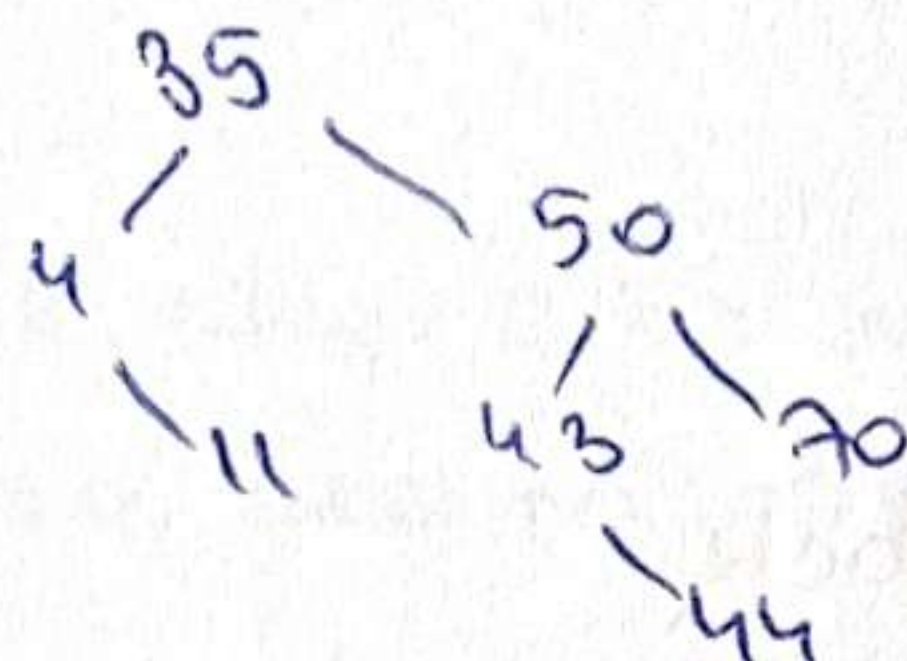


BT 5 - insert: 40, 4, 35, 11, 50, 70, 43, 44.
 - remove 40.

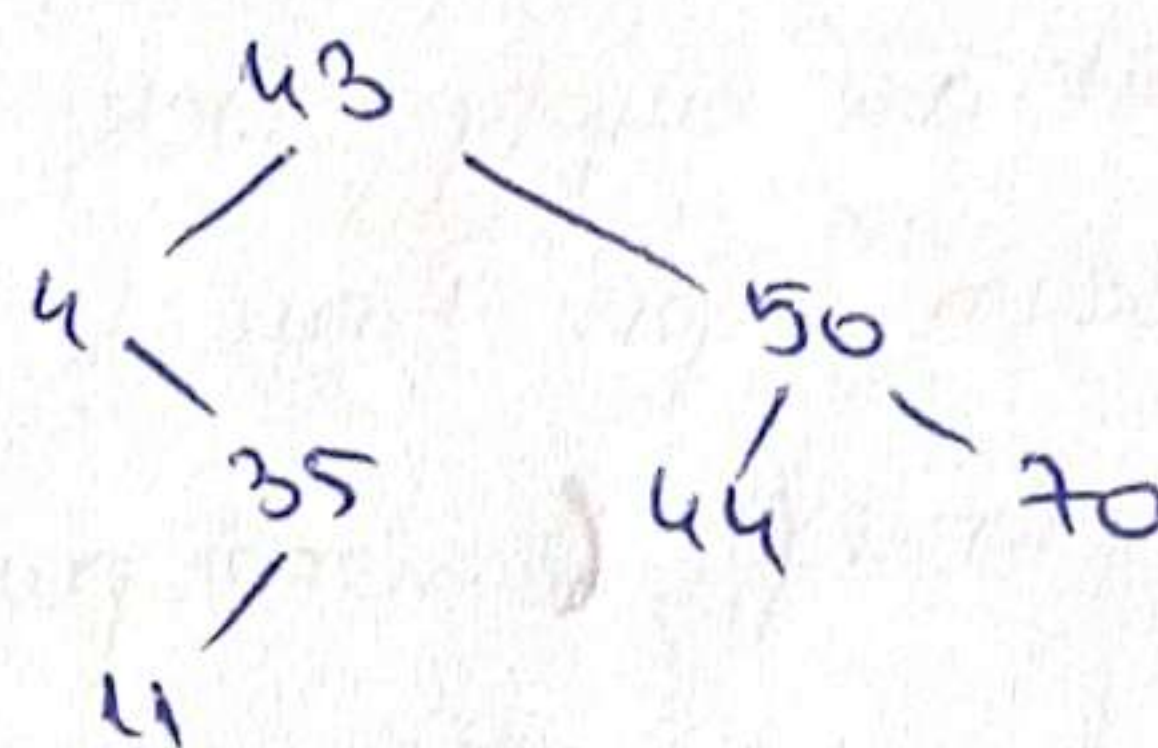


i. replace w predecessor: (35)

2)



ii. w successor (43)



c) HT, $m=11$, collision res. coalesced chaining

insert: 7, 84, 13, 33, 76, 29, 12, 98

remove 7

$$h(7) = 7 \% 11 = 7 \checkmark$$

$$h(84) = 7 \checkmark$$

$$h(13) = 2 \checkmark$$

$$h(33) = 0 \checkmark$$

$$h(76) = 10 \checkmark$$

$$h(29) = 7 \checkmark$$

$$h(12) = 1 \checkmark$$

$$h(98) = 10$$

pos	0	1	2	3	4	5	6	7	8	9	10
T	84	29	12	7		33	13	7			76
next	X	X	X	X	-	-	-	-	-	-	-
	1	2	3	4							

first empty: $\emptyset$ X X X X

pos	0	1	2	3	4	5	6	7	8	9	10
T	84	29	13	29	12	98		7			76
next	X	X	-	X	X	X	-	0	-	-	-
	1	3		4	5	6					

first empty: $\emptyset$ X X X X X

Case iii of removal. ~ If some elem. hashes to the position we

are deleting (7), move it there, then continue deletion from the old pos

→ remove 7 from pos 7; 84 hashes to 7 ⇒ move 84 at pos 7.

→ remove old next(0); 33 hashes to pos 0 ⇒ move 33 at pos 0.

→ remove old next(1); 12 hashes to pos 1 ⇒ move 12 at pos 1.

old (4); no one hashes there ⇒ delete pos 4.

⇒ final:

P	0	1	2	3	4	5	6	7	8	9	10
T	33	12	13	29	-	98	-	84	-	-	76
n	1	3	-	4	-	-	-	0	-	-	5

1.) From which traversals can we rebuild any BT?

- a) preorder + inorder
- b) inorder + postorder

2) Max. nr. of parentheses on the stack?

$((()((()((())))$

Balanced seq. of parentheses algorithm:

1) init. an empty stack

2) scan expr. from L $\rightarrow$ R

$\hookrightarrow$ if ($\Rightarrow$ push onto the stack

$\hookrightarrow$ if)

$\hookrightarrow$ if stack empty $\Rightarrow$ invalid seq.

$\hookrightarrow$ else pop one from the stack $\rightarrow$ check type of parenthesis

$\hookrightarrow$ if stack empty $\Rightarrow$ valid

$\hookrightarrow$ else $\Rightarrow$ invalid

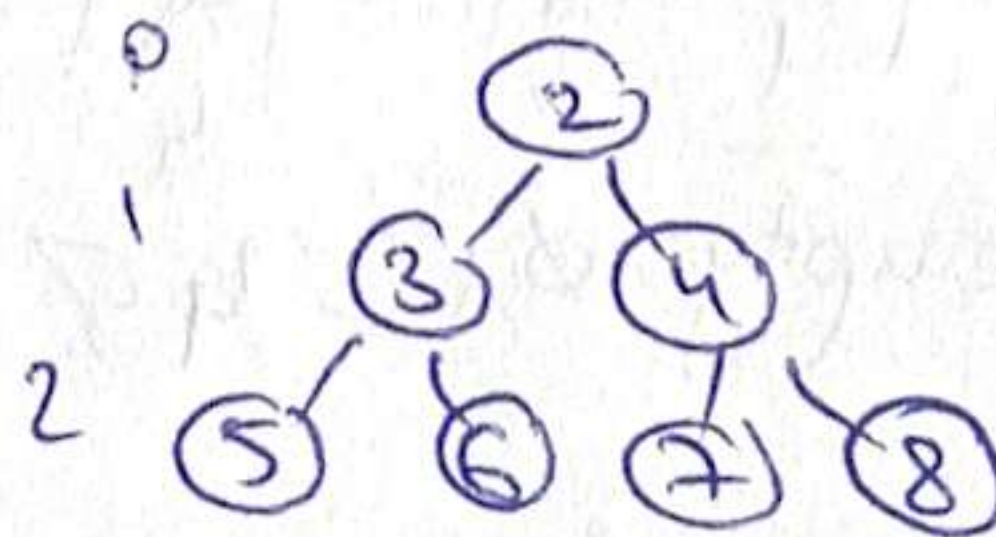
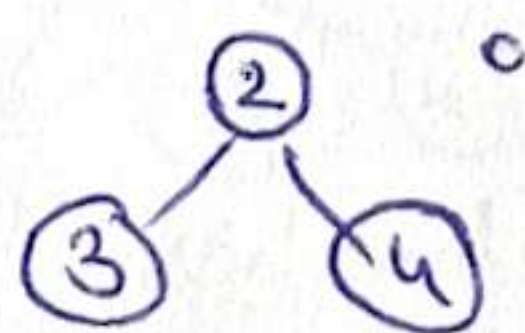
Stack: ((((((((

)))))))) $\checkmark$ valid.

$\Rightarrow$ 0 3

3) $\alpha = \frac{m}{m} > 1 \Leftrightarrow m > m$ possible only with e separate chaining

4) Bin. min. heap with > 1000 nr.



a. 2nd smallest: level 1 T.

b 3rd smallest: F (level 2).

c. 4th smallest: level 2. T

d. 5th smallest: (the pic. is out)

e. 6th smallest

subalgorithm init(q) is

$q.rear \leftarrow Nil$

function isEmpty(q) is

isEmpty $\leftarrow q.rear = Nil$

$\rightarrow$ add to the rear

subalgorithm enqueue(q, e) is

newNode $\leftarrow$ allocate

[newNode].info $\leftarrow e$

if $q.rear = Nil$ then // if q is empty
[newNode].next $\leftarrow$ newNode // circular list $\Rightarrow$ point to itself
// the only node in the list
 $q.rear \leftarrow$ newNode // only node $\Rightarrow$ both front and rear

else

[newNode].next $\leftarrow$ [q.rear].next // point to front node
[q.rear].next $\leftarrow$ newNode
 $q.rear \leftarrow$ newNode

function dequeue(q) is

if $q.rear = Nil$ then // q empty
@throw ...

front $\leftarrow$ [q.rear].next

dequeue $\leftarrow$ [front].info

if front = $q.rear$ then

$q.rear \leftarrow Nil$

else

[q.rear].next $\leftarrow$ [front].next // skip the front

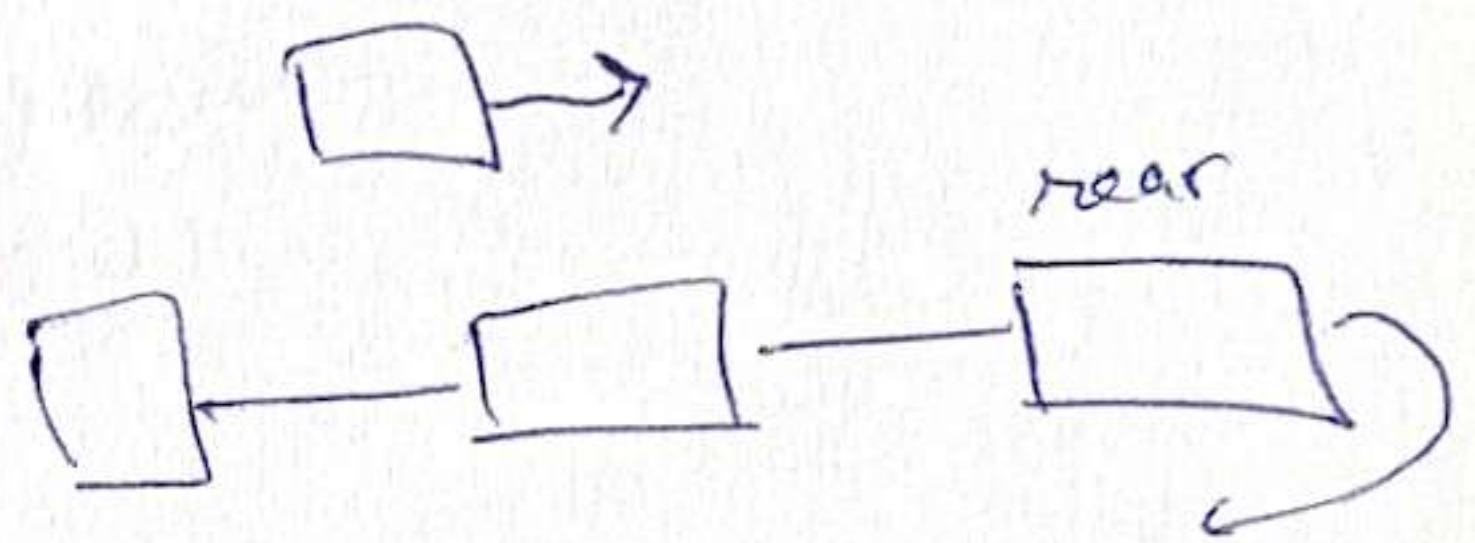
deallocate front

function top(q) is

if $q.rear = Nil$ then

@throw ...

top $\leftarrow$ [q.rear].next.info



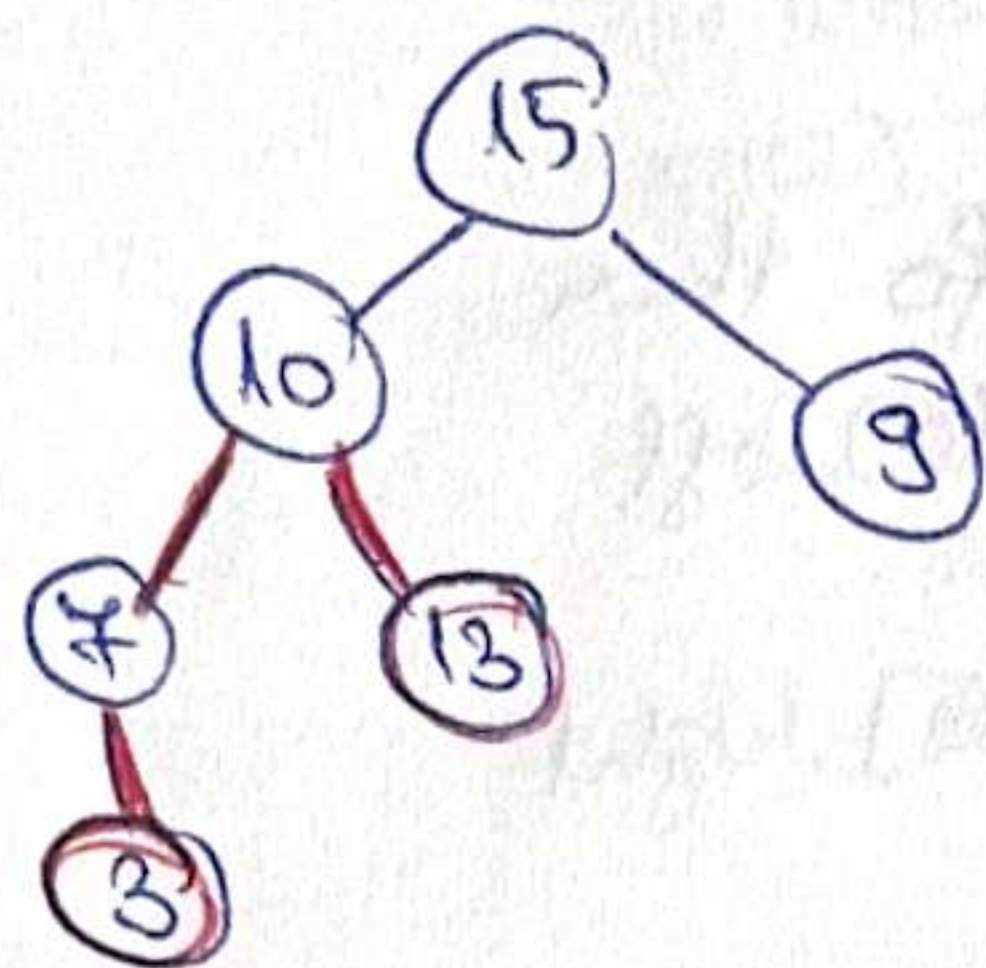
1) BST

e_1, e_2 may/may not be in BST.

length of the shortest path between e_1 and e_2 .

if either $\nexists e_1$ or $\nexists e_2 \Rightarrow$ return -1.

✓ repres. of BST (linked repes, dyn. alloc, without parent/height)
implement op $\rightarrow$ total complexity.



LCA = 10

distance (10, 3) = 2

distance (10, 13) = 1

$\Rightarrow$ shortest path = 2 + 1 = 3

ex: 3, 13 $\Rightarrow$ length = 3

least common ancestor (here, 10).

a)

Node

info: TElem

left: $\uparrow$ Node

right: $\uparrow$ Node

BST:

root: $\uparrow$ Node

b)

helper $\rightarrow$ lowest common ancestor (BST) = the deepest / lowest node that has both e_1 and e_2 as descendants.

function LCA(node, e_1, e_2) is

while node $\neq$ Nil ex

if $e_1 < [\text{node}].\text{info}$ and $e_2 < [\text{node}].\text{info}$ then

node $\leftarrow [\text{node}].\text{left}$ // move down left

else if $e_1 > [\text{node}].\text{info}$ and $e_2 > [\text{node}].\text{info}$

node $\leftarrow [\text{node}].\text{right}$

else // we found it (when the paths diverge; one goes left, one goes right!)

LCA $\leftarrow$ node

return

LCA $\leftarrow$ Nil (one or both $\nexists$)

Bc: $\Theta(1)$

wc: $\Theta(h)$

} single root-to-leaf path.

helper - distance between two nodes

function distance (node, x) is

$d \leftarrow 0$

while node $\neq$ Nil ex

if $x = [\text{node}].\text{info}$ then

distance $\leftarrow d$

return

if $x < [\text{node}].\text{info}$ then

node $\leftarrow [\text{node}].\text{left}$

else

node $\leftarrow [\text{node}].\text{right}$

$d \leftarrow d + 1$

distance $\leftarrow -1$

Bc: $\Theta(1)$ } single root-to-
wc: $\Theta(h)$ } leaf path

required op:

function shortestPath (tree, e_1 , e_2) is

ancestor $\leftarrow \text{LCA}(\text{tree.root}, e_1, e_2);$

if ancestor = Nil then

shortestPath $\leftarrow -1$

return

$d_1 \leftarrow \text{distance}(\text{ancestor}, e_1)$

$d_2 \leftarrow \text{distance}(\text{ancestor}, e_2)$

if $d_1 = -1$ or $d_2 = -1$ then

shortestPath $\leftarrow -1$

else

shortestPath $\leftarrow d_1 + d_2$

Bc: $\Theta(1)$ } $h-1$
wc: $\Theta(h)$

T: $O(h)$

h = height of BST

$h = \log_2 n$ if BST balanced

$h = n$ if BST skewed

